

☒ Session 17: Exploit Analysis Automation & Debugging Basics

☒ Part 1: Exploit Analysis Automation Process

☒ What is Exploit Analysis?

Exploit analysis is the process of examining how a vulnerability is exploited to:

- Understand the nature of the exploit
 - Determine the risk/severity
 - Recreate and automate testing or patching processes
-

❏ Why Automate Exploit Analysis?

Manual analysis is slow and prone to error. Automation helps in:

- Efficient detection of known exploits
- Repetitive testing using fuzzers
- Scalable scanning across systems
- Report generation for vulnerabilities

❏ Automation Tools & Techniques

Tool	Purpose
------	---------

Metasploit	Exploit testing, payload delivery
Boofuzz	Fuzzing automation
pwntools	Scripting exploitation in Python
Ghidra/IDA	Reverse engineering with scripting
AutoSploit	Mass-exploit known CVEs
Nmap NSE	Scriptable exploit detection

☒ Process Workflow

1.
Input collection (target IPs, binaries)
2.
Vulnerability scanning (e.g., Nmap, Nessus)
3.
Auto-exploitation (Metasploit, AutoSploit)

4.

Logging/reporting

5.

Custom scripting using Python or bash

☒ Sample Python Script Using pwntools

python

CopyEdit

```
from pwn import * p = process('./vuln_app') payload = b'A' * 64 + b'\x90\x90...' p.sendline(payload) p.interactive()
```

☒ Part 2: Debugging Basics

⌘ What is Debugging?

Debugging is the process of identifying, isolating, and fixing bugs in software or systems.

⌘ Types of Debugging

- Static Debugging: Without running code (code review)
 - Dynamic Debugging: During runtime (using debugger)
-

⌘ Basic Debugging Concepts

Term	Description
Breakpoint	Stops execution at a given line
Step Over	Executes current line and moves ahead

Step Into	Jumps inside function call
Watch Variable	Monitor variable value at runtime
Stack Trace	Function call history

☒ Tools

- GDB – GNU Debugger (Linux)
- PDB – Python Debugger
- OllyDbg – Windows binary debugger
- x64dbg – Windows x64 debugger

- **VSCode** – For Python/JavaScript/etc.

☒ Example: Debugging Python Code

python

CopyEdit

```
import pdb
def divide(x, y):
    pdb.set_trace()
    return x / y
divide(10, 0)
```

Output:

scss

CopyEdit

(Pdb) l (Pdb) p x (Pdb) p y (Pdb) n
